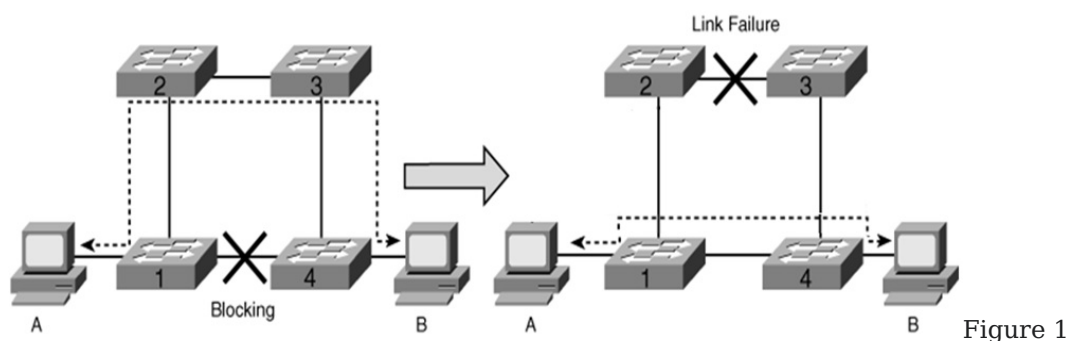




STP Attack and Defense

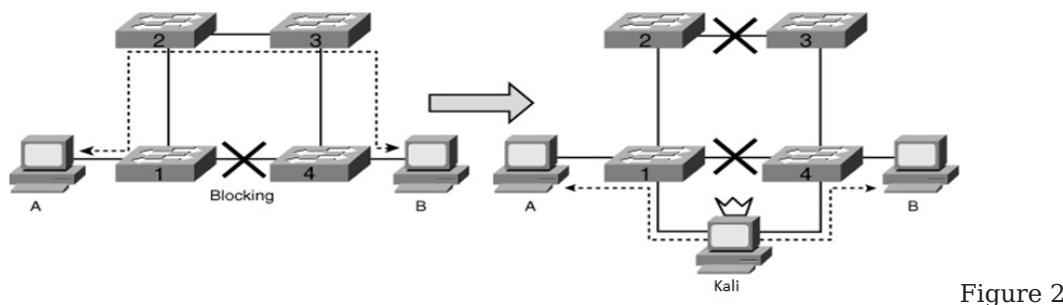
To achieve network availability, switches are commonly interconnected with redundant links and IEEE 802.1w **Rapid Spanning Tree Protocol (RSTP)** is automatically run to prevent loops. An example is shown in Figure 1 where 4 switches are interconnected with redundant links forming a loop, but the link between switches 1 and 4 was initially blocked as a result of RSTP running.



In the event of a link failure, say between switches 2 and 3, switches are able to re-configure themselves due to the benefit of **RSTP** so that the link between switches 1 and 4 will become forwarding and communications between PC A and B are able to continue without much disruption.

Conducting STP Attack

Unfortunately, RSTP does not have security. Figure 2 shows a scenario when an attacker connects his PC to switches 1 and 4 to inject **RSTP BPDUs** (bridge protocol data unit) with **lower bridge priority** to take over as **root bridge**. As a result, all network traffic between PC A and B will now be forwarded via the attacker's PC and be sniffed by the attacker!

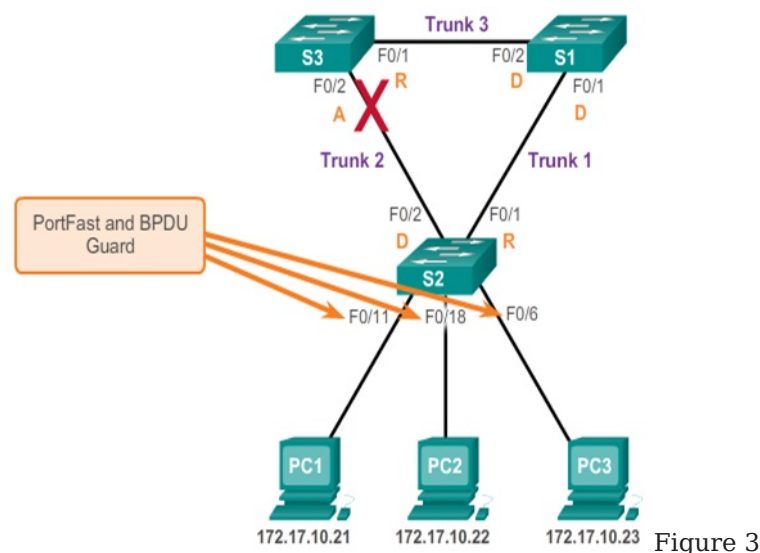


The procedure for an attacker using Kali to conduct STP attack is illustrated in the lab exercise which you will try out in the lab class.

Defending STP Attack

To **defend** against **STP attack**, **BPDUs guard** can be enabled on every switch interface that is not supposed to receive RSTP BPDUs. For example in Figure 3, interfaces F0/6, F0/11 and F0/18 connected to user PCs could be enabled with BPDUs guard using the commands:

```
S2(config)# interface interface-id
S2(config-if)# spanning-tree bpduguard enable
```



In practice when you connect a PC to a switch port, you may have noticed a time lag before the switch port is operational due to the need to transit through different RSTP states as shown in Figure 4 to prevent network loops.

802.1D State	802.1w State	Forwards Data Frames?	Learns MACs Based on Received Frames?	Transitory or Stable State?
Disabled	Discarding	No	No	Stable
Blocking	Discarding	No	No	Stable
Listening	Not used	No	No	Transitory
Learning	Learning	No	Yes	Transitory
Forwarding	Forwarding	Yes	Yes	Stable

Figure 4

To avoid the time lag, interfaces connected to the user PCs can be configured with **portfast** to transit to forwarding state directly since they will not form network loop by design.

```
S2(config)# interface interface-id
S2(config-if)# spanning-tree portfast
```

In this case, a more direct way to enable **BPDU guard** is to configure it globally on all **portfast-enabled ports** using the command:

```
S2(config)# spanning-tree portfast bpduguard default
```

Once a switch port is enabled with **BPDU guard**, it will become **error-disabled** when a RSTP BPDU is received at that port, thus preventing STP attack.

To **manually recover** a switch port from the error-disabled state, configure:

```
S2(config)# interface interface-id
S2(config-if)# shutdown
S2(config-if)# no shutdown
```

Alternatively, a switch port may be configured to **recover automatically** after a time interval using the commands:

```
S2(config)# errdisable recovery cause violation-reason
S2(config)# errdisable recovery interval time-in-seconds
```

Another **defense** against STP attack is the **root guard** feature which enforces root bridge placement in the network. By design, distribution switches are typically primary and secondary root bridges, and root guard may be configured on their interfaces connected to the access switches as shown in Figure 5.

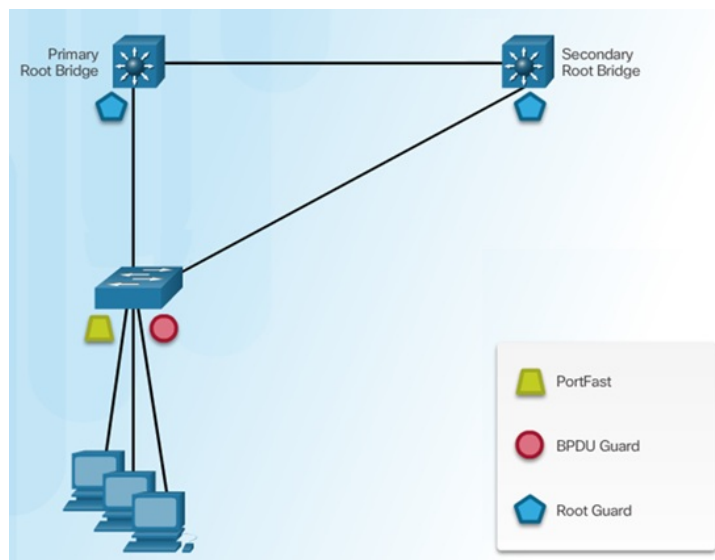


Figure 5

In contrast to BPDU guard, a switch port configured with **root guard** is able to receive BPDU as long as this is not superior BPDU to take over the root. If a superior BPDU is received, the switch port will transit temporarily to root-inconsistent state and block all traffic until the superior BPDU stops. In this way, access switches will not be able to become root and RSTP is able to continue running.

In general, switch ports that will always be **designated ports** and **not** to become **root ports** may be configured with **root guard** using the commands:

```
switch(config)# interface interface-id
switch(config-if)# spanning-tree guard root
```

Reference: [Cisco - Enhance STP with Root Guard](#)